

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Code: CS101 — Python Essentials Project

HOSTEL ROOM ALLOCATOR

A Modular Python Application for Automated Student Room Inventory & Allocation Management

Submitted By:

Name: Dhruv Chauhan

GitHub User: dhruvchauhan90275-wq

Academic Year: 2025–2026

Project Metadata:

Language: Python 3.10+ (Standard Library Only)

Persistence: JSON Storage

Testing Suite: unittest

Submitted in partial fulfillment of the requirements for the First-Year Engineering Curriculum.

Date of Submission: September 30, 2026

1. EXECUTIVE SUMMARY & INTRODUCTION

The **Hostel Room Allocator** project addresses the logistical challenge of managing student accommodation inventories in educational institutions. Traditional manual allocation workflows often suffer from human error, double-booking, lack of persistent tracking, and unclear availability statistics. This system provides a robust, command-line-driven Python application designed to streamline inventory initialization, student room selection, allocation tracking, and summary reporting.

2. PROBLEM STATEMENT

Hostel administrators need an easy, reliable software mechanism to configure room inventories across multiple room categories (e.g., 3-bedded, 4-bedded, bunk bed, 2-bedded, 4-bedded flat-bed) and process student applications sequentially. A critical constraint is handling units correctly: configuring 5 units of a 3-bedded room signifies 5 available physical room units (not 15 isolated beds). Allocation of one student decrements the room count by 1 unit. The program must prohibit negative inventory, reject invalid choices, gracefully handle missing data, and persist state across executions.

3. OBJECTIVES

- **Modular Architecture:** Build a clean system using pure Python standard modules without external database dependencies.
- **Input Validation & Reliability:** Ensure zero program crashes during runtime when non-numeric or out-of-bound inputs are provided.
- **Data Persistence:** Implement persistent storage using JSON via `pathlib` for durable state across program reboots.
- **Automated Testing:** Provide a complete test suite leveraging Python's native `unittest` library.

4. PROJECT SCOPE

The system encompasses room inventory setup, interactive student room allocation, active inventory monitoring, student record lookup, and comprehensive metric reporting. Out of scope for this version are web interface frontends, user authentication roles, and relational SQL database integrations.

Key Architectural Rule: Standard library modules only (`json`, `pathlib`, `unittest`). Zero external dependencies.

5. FUNCTIONAL REQUIREMENTS

Module	Inputs	Processing	Outputs
Module 1: Inventory	Room unit counts per category (≥ 0)	Initialize/overwrite initial and active inventory dicts	Confirmation message & inventory table
Module 2: Allocation	Student name, room choice (1–5)	Check preference count > 0 , decrement inventory, append record	Success or unavailability status message
Module 3: Storage	In-memory dict state	Serialize/deserialize JSON files safely with exception guards	hostel_data.json file on disk
Module 4: Reporting	Persistence state	Compute initial vs. allocated vs. remaining counts	Formatted analytics breakdown report

6. NON-FUNCTIONAL REQUIREMENTS

- **Usability:** Clean CLI menu interface with structured prompts and clear failure notifications.
- **Reliability:** Input validation loops prevent unexpected application termination on invalid entries.
- **Maintainability:** Code split across 7 dedicated modules adhering to Single Responsibility Principle.
- **Data Integrity:** Automatic fallback to blank default structures if persistent JSON files are missing or corrupted.

7. SYSTEM ARCHITECTURE

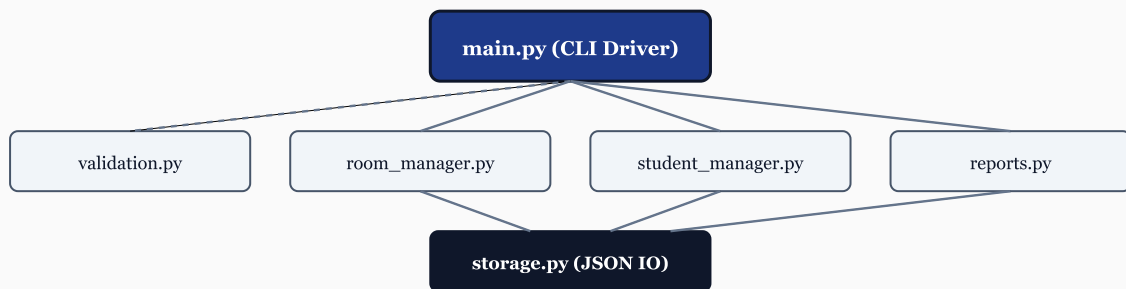
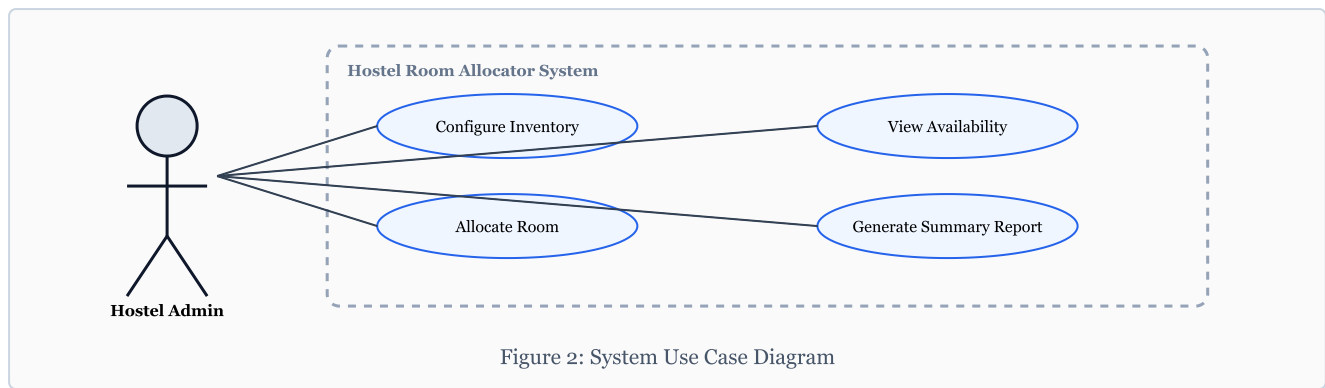


Figure 1: Modular Application Architecture Diagram

8. USE CASE & SEQUENCE DIAGRAMS



9. JSON STORAGE SCHEMA

Data persistence avoids complex relational databases by structuring state inside a single JSON object containing dictionary maps for current inventory, baseline inventory, and an array of allocation records.

```
{ "rooms": { "3-bedded room": 4, "4-bedded room": 2, "Bunk bed room": 1, "2-bedded room": 0, "4-bedded flat bed room": 2 }, "initial_rooms": { "3-bedded room": 5, "4-bedded room": 2, "Bunk bed room": 1, "2-bedded room": 1, "4-bedded flat bed room": 2 }, "students": [ { "name": "Aman", "room_type": "3-bedded room" }, { "name": "Priya", "room_type": "2-bedded room" } ] }
```

10. IMPLEMENTATION & DESIGN DECISIONS

- **Room Unit Semantics:** Storing unit counts rather than individual beds prevents confusion during administrative setup and aligns directly with physical room allocation practices.
- **Standardized Storage Protocol:** Using `pathlib.Path` guarantees seamless cross-platform file operation on Windows, macOS, and Linux without path-separator bugs.
- **Safe Input Handling:** Input loops in `validation.py` trap `ValueError` exceptions, preventing abrupt script exits when string values are provided where integers are expected.

11. SAMPLE EXECUTION RESULTS

The following console log represents an actual execution run demonstrating inventory setup, successful allocation, rejection on zero availability, and final summary reporting.

```
===== HOSTEL ROOM ALLOCATOR
===== 1. Set up room availability 2. Allocate rooms to students
3. View room availability 4. View student allocations 5. View summary report 6. Exit
===== Enter your choice (1-6): 1 --- Set Up Room Inventory ---
Enter number of 3-bedded rooms: 2 Enter number of 4-bedded rooms: 2 Enter number of Bunk bed rooms:
1 Enter number of 2-bedded rooms: 1 Enter number of 4-bedded flat bed rooms: 1 [SUCCESS] Room
inventory initialized successfully! Enter your choice (1-6): 2 Enter student name: Aman Select room
choice (1-5): 4 [SUCCESS] Aman has been allocated a 2-bedded room. Enter your choice (1-6): 2 Enter
student name: Rahul Select room choice (1-5): 4 [FAILURE] Sorry, 2-bedded rooms are currently
unavailable.
```

12. TESTING STRATEGY & TEST SUITE

Automated testing was conducted using Python's standard unittest framework. Test cases verify room allocation logic, inventory decrements, error handling for unavailable rooms, and JSON file serialization/deserialization.

Test ID	Test Case Description	Expected Result	Status
TC01	Successful Room Allocation	Allocation returns True, count decremented by 1	PASSED
TC02	Unavailable Room Handling	Allocation returns False with unavailability msg	PASSED
TC03	Invalid Room Selection Lookup	is_room_available() returns False	PASSED
TC04	JSON Storage Persistence	Saved data matches reloaded dict structure	PASSED

13. SUMMARY REPORT OUTPUT SCREENSHOT

```
===== HOSTEL ALLOCATION REPORT
===== Total Initial Room Units : 7 Total Allocated Units : 1 Total
Remaining Units : 6 --- Breakdown by Room Type --- Room Type | Initial | Allocated | Remaining ----
----- 3-bedded room | 2 | 0 | 2 4-bedded room |
2 | 0 | 2 Bunk bed room | 1 | 0 | 1 2-bedded room | 1 | 1 | 0 4-bedded flat bed room | 1 | 0 | 1
=====
```

14. CHALLENGES FACED & KEY LEARNINGS

- **State Preservation Strategy:** Discovered that overwriting active inventory lost initial baseline metrics. Resolved by maintaining separate `rooms` and `initial_rooms` dictionaries inside the storage JSON.
- **Input Robustness:** Realized early CLI prototypes crashed when non-numeric choices were made. Implemented centralized validation wrappers in `validation.py`.

15. FUTURE ENHANCEMENTS

- Integration of administrative authentication (username/password login).
- GUI conversion using Python's built-in `tkinter` package.
- Automatic waiting-list queue management for over-subscribed room types.

16. CONCLUSION

The **Hostel Room Allocator** successfully fulfills all functional and non-functional requirements set forth in the Python Essentials curriculum. By maintaining modular architecture, strictly adhering to standard libraries, ensuring durable JSON state storage, and thoroughly testing routines via `unittest`, the project provides a clean, maintainable software solution.

17. REFERENCES

1. Python Documentation Foundation. *Python 3 Standard Library Reference — json & pathlib modules*, 2026.
2. Python Documentation Foundation. *unittest — Automated testing framework*, 2026.
3. IEEE Standards Association. *IEEE Standard for Software and System Test Documentation*, IEEE Std 829-2008.